LLMOps Assignment 2: API Cost, Latency and Prompt Versions

EduAssist AI Q&A assistant. Repository: https://github.com/sreejarepo/LLMOps

1. Aim

I act as the LLMOps engineer for EduAssist AI's Q&A assistant. I measure latency and token cost, compare three prompt versions, and promote the best one in MLflow. Unlike plain MLOps, prompts and cost per request are tracked as production artifacts.

2. Setup and why I used a local model

My OpenAI calls failed with an insufficient_quota error, so I followed section 7 of the brief and used an open-source model instead: llama3.1:8b, served by Ollama on a Tesla T4 GPU. I call it through Ollama's OpenAI-compatible endpoint with the openai library at temperature 0, so no API key is needed. Because local inference is free, I estimated cost from DeepInfra's listed price for the same model on OpenRouter, checked on 2026-11-04: 0.02 USD per million input tokens and 0.04 USD per million output tokens.

3. Task 1: latency

I ran prompt v2 on five questions. Latency in seconds: mean 10.52, median 10.63, minimum 7.72, maximum 13.26, standard deviation 2.45. The slowest request was Q5 at 13.26 s with 426 output tokens, while the other four averaged 9.84 s and 342 output tokens. Latency and output length are strongly linked (r = 0.98) at roughly 34 tokens per second, since tokens are generated one after another. Other possible causes are prompt length and GPU scheduling.

4. Task 2: tokens and cost

The average cost was 0.0000154 USD per request, so 10,000 requests a month would cost about 0.15 USD. Output tokens made up 93% of the cost, because they are priced twice as high as input tokens. My recommendation is to shorten responses. Capping output at 200 tokens would cut the average from 359 to about 200 tokens and the cost per request to 0.00000905 USD, which is 41% less. I would check answer quality first.

5. Task 3: prompt versions in MLflow

I registered the three prompts as eduassist_qa_prompt versions 1 to 3 in the experiment eduassist_llmops_assignment and ran each on the same five questions. The table shows the averages.

Version	Avg latency (s)	Avg input tok	Avg output tok	Avg cost (USD)	Valid JSON	Recommended?
v1	16.91	30.6	500.4	0.00002063	0%	No
v2	13.42	52.6	358.6	0.00001540	0%	No
v3	4.48	86.6	115.2	0.00000634	100%	Yes

6. Why I promoted v3

I promoted v3 (registry version 3) to the production alias. My rule ranked the valid JSON rate first and average cost second. Only v3 returned valid JSON, 100% against 0% for v1 and v2, which matters because downstream code must parse every response.

v3 was also the fastest and cheapest. Its prompt is longer (86.6 input tokens against 52.6 for v2), but it produced far fewer output tokens (115 against 359), and those drive generation time and cost. Latency fell from 13.42 s to 4.48 s and cost per request from 0.0000154 to 0.00000634 USD.

On quality, I read all 15 answers and rated each for correctness (answer_quality_check.csv). v1 wrongly says in Q4 that threads share the same stack and program counter. v2 was the most accurate (4 correct, 1 partly correct). v3 was correct on 3 of 5 but shallower (44-word answers, 3.0 key points), and its Q5 says the goal is "low bias and low variance", which overstates what the tradeoff allows. All five v3 answers say confidence

"high", so that field is not calibrated. I accept this small loss of depth for output that always parses.

So the JSON guarantee cost nothing extra here. One caution: Task 1 and Task 3 both ran v2 with the same token counts, yet latency was 10.52 s and 13.42 s, so timing varies between runs. Five samples also cannot prove JSON stays reliable at scale, so I would keep monitoring it and keep v1 and v2 for rollback.

7. Evidence (click to open)

- Screenshot 1: experiment runs with metrics
- Screenshot 2: metrics compared across runs
- Screenshot 3: Prompt Registry, v1 to v3
- Screenshot 4a: prompt diff, v1 vs v2
- Screenshot 4b: prompt diff, v2 vs v3
- Screenshot 5: production alias on v3
- Pricing evidence (DeepInfra on OpenRouter)
- Notebook
- Notebook as PDF
- latency_tokens_cost.csv
- prompt_evaluation_results.csv
- answer_quality_check.csv
- promotion_justification.txt
- README